BRIXCOT Lead Generation Platform - System Architecture & Data Models

Document 1 of 3: Core Architecture, User Roles, and Data Schema

1. APPLICATION PURPOSE & ARCHITECTURE

Overview

BRIXCOT is a sophisticated **B2B Lead Generation and Distribution Platform** built with **Next.js 16**, designed to connect lead sellers with lead buyers. The platform facilitates lead capture, qualification, distribution, monetization, and CRM integration.

Tech Stack

- **Frontend**: Next.js 16 (Turbopack), React 19, Redux Toolkit, Material-UI (MUI v7)
- **Backend**: Next.js API Routes, Node.js
- **Database**: MongoDB with Mongoose ODM
- **Authentication**: NextAuth.js with Google OAuth and Email providers
- **Caching**: Redis (Upstash) for session management
- **Payment Processing**: Stripe (Connect for marketplace), PayPal, Square
- **Communication**: Twilio (SMS/Voice), Nodemailer (Email)
- **Real-time**: Socket.io
- **Integrations**: Zapier, HubSpot, Salesforce

Architecture Pattern

- **Multi-tenant SaaS** with role-based access control (RBAC)
- **Marketplace model** with platform fees and connected accounts
- **Event-driven** with webhook-based integrations
- **Microservices-oriented** API structure

- **Cache-first** strategy with Redis for performance optimization

2. USER ROLES & CAPABILITIES

A. SELLER (Lead Generator)

Primary user who generates and sells leads

Core Capabilities:

- **Form Management**: Create and manage customizable lead capture forms
- **Call Tracking**: Configure tracking phone numbers with Twilio integration
- **Buyer Management**: Register, configure, and assign lead buyers
- **Pricing Control**: Set lead pricing (per-unit or per-lead basis)
- **Distribution Methods**:
  - **Manual**: Seller manually assigns leads to specific buyers
  - **Round Robin**: Automatic equal distribution across buyers
  - **Weighted**: Distribution based on buyer priority weights (1-10)
  - **Priority**: Assigned to highest priority buyer first
- **Financial Management**: Process payments and manage wallet balance
- **Payment Integration**: Configure Stripe Connected Account for direct payments
- **Marketing**: Create and manage Email/SMS campaigns
- **Automation**: Set up workflow automations based on triggers
- **CRM Integration**: Configure HubSpot, Salesforce, and Zapier integrations
- **Invoicing**: Generate, send, and track invoices to buyers
- **Analytics**: View comprehensive reports and dashboard metrics
- **Subscription**: Manage tiered subscription plans

Subscription Features:

- Tiered access to features (forms, leads, buyers, Twilio numbers)
- Call recording and tracking capabilities
- Live chat support (Tawk.to integration)
- Import/export capabilities
- Advanced analytics and reporting

File References:

- Dashboard: `app/dashboard/seller/`
- APIs: `app/api/leads/` , `app/api/buyers/` , `app/api/form/`

B. BUYER (Lead Purchaser)

User who purchases leads from sellers

Core Capabilities:

- **Lead Browsing**: Browse marketplace leads (status: "available", non-exclusive)
- **Lead Purchase**: Buy leads using units/credits
- **Lead Management**: Accept or reject assigned leads
- **Wallet Management**: Manage balance and purchase credit units
- **Payment Methods**: Purchase units via Stripe or PayPal
- **Preference Configuration**:
  - **Service Locations**: Define cities, states, zip codes, and search radius
  - **Industries**: Select relevant industry categories
  - **Lead Types**: Choose exclusive vs. shared leads
  - **Price Thresholds**: Set maximum price per lead
  - **Daily Limits**: Configure maximum leads per day
- **Criteria Sets**: Create automated lead matching rules
- **Lead History**: View assigned and purchased leads
- **Notifications**: Receive alerts via email, SMS, or dashboard
- **Quality Feedback**: Rate lead quality and provide feedback
- **Call Access**: Listen to call recordings (if enabled by seller)
- **Schedule Management**: Configure working hours and timezone

Lead Matching Algorithm:

- **Location-based**: Strict or radius-based matching
- **Industry filtering**: Match against buyer's selected industries
- **Auto-accept**: Automatically purchase matching leads
- **Source exclusion**: Block specific lead sources

File References:

- Dashboard: `app/dashboard/buyer/`

- APIs: `app/api/leads/available` , `app/api/buyers/`

C. ADMIN (Platform Administrator)

Platform owner with full system access

Core Capabilities:

- **User Management**: Manage all users (sellers, buyers, staff)
- **Tier Management**: Create and configure subscription tiers
- **Tier Configuration**: Set limits, pricing, and features
- **Financial Oversight**: View all transactions and invoices
- **Content Management**: Manage help content (FAQs, videos)
- **System Monitoring**: Monitor system health and performance
- **Support Management**: Review and resolve support tickets
- **Manual Adjustments**: Adjust wallet balances and units
- **Analytics**: Access platform-wide analytics and reports

File References:

- Dashboard: `app/admindashboard/`
- APIs: `app/api/admin/`

D. BUSINESS-ADMIN & STAFF

Supporting roles for enterprise features

Capabilities:

- Similar to seller with organizational hierarchy
- Manage team members
- Access shared resources
- Role-based permissions within organization

3. DATA MODELS & RELATIONSHIPS

Core Database Schema

1. User Model

File: `models/userModel.ts`

```
{
  // Basic Info
  _id: ObjectId
  username: string
  name: string
  email: string (unique, indexed)
  businessName?: string
  loginlink?: string

  // Authentication
  role: "admin" | "seller" | "buyer" | "user" | "business-admin" | "staff"
  provider: "google" | "github" | "credentials"
  image?: string
  emailVerified?: boolean
  status: "active" | "suspended"

  // Financial
  walletBalance: number (default: 0)

  // Seller-specific Fields
  leadDistributionMethod?: "manual" | "roundRobin" | "weighted" | "priority"
  buyers: ObjectId[] // References to Buyer documents
  leads: ObjectId[] // References to Lead documents
  forms: ObjectId[] // References to Form documents

  // Twilio Integration
  twilioActivated?: boolean
  twilioAccountSid?: string (encrypted)
  twilioAuthToken?: string (encrypted)
  trackingNumbers: [{
    phoneNumber: string
    industry: string
    forwardingType: "single_multiple" | "specific_lead"
    method: "live_call" | "voicemail"
    recordCall: boolean
    welcomeMessage?: string
    callWhisper?: string
    requireResponse?: boolean
    forwardingNumbers: string[]
    leadBuyers: ObjectId[]
    buyerResponses: Map<string, string>
  }]
```

```
// Email Settings
emailSettings?: {
  emailAddress: string
  smtpServer: string
  smtpUser: string
  smtpPassword: string (encrypted)
  port: number
}

// Payment Integration
stripeAccountId?: string
stripeOnboarded?: boolean
stripeCustomerId?: string
paypalCustomerId?: string
tosAcceptance?: {
  accepted: boolean
  acceptedAt: Date
  ipAddress: string
}

// Subscription Management
subscription: {
  subscriptionTierId: ObjectId (ref: Tier)
  isSubscriptionActive: boolean
  subscriptionStartDate?: Date
  subscriptionExpiryDate?: Date
  subscriptionPaymentMethod?: "stripe" | "paypal"
  autoRenew: boolean (default: true)
  notificationsSent: number (default: 0)
  lastNotificationDate?: Date
}

// Credit System
creditSetup?: {
  stripeKeys?: { publishableKey, secretKey }
  paypalKeys?: { clientId, secretKey }
}
unitPricingOptions?: [{
  unitPrice: number
  minUnits: number
  maxUnits: number
}]
callChargeOptions?: {
```

```
    chargePerMinute: number
    freeMinutes: number
  }

  // Timestamps
  createdAt: Date
  updatedAt: Date
}
```

Indexes:

- `email` (unique)
- `role`
- `subscription.subscriptionTierId`
- `subscription.subscriptionExpiryDate`

2. Lead Model

File: `models/leads.ts`

```
{
  _id: ObjectId

  // Basic Information
  name: string (required)
  email: string (indexed)
  phone: string
  company?: string
  conversationId?: string

  // Qualification Scoring
  leadScore: number (0-10, default: 5)
  scoreFactors: {
    completeness: number (0-3)
    responsiveness: number (0-3)
    valuePotential: number (0-4)
  }
  qualificationScore: number (0-100, default: 0)

  // Ownership & Association
  userId: ObjectId (ref: User - seller) (required, indexed)
  formId: ObjectId (ref: Form)
  calls: ObjectId[] (ref: Call)

  // Custom Fields (form-specific data)
  fields: [{
    id: string
    label: string
    value: any
  }]

  // Status & Distribution
  status: "new" | "available" | "sold" | "assigned" | "qualified" | "unqualified" | "transferre
  distributionMethod?: "manual" | "round_robin" | "marketplace"

  // Exclusivity & Sharing
  exclusive: boolean (default: false)
  shared: boolean (default: false)
  shareNumber?: number // Max times lead can be sold
  soldCount: number (default: 0) // Times lead has been sold
  unit?: number // Price in units

  // Assignment Tracking
```

```
  assignedTo: [{
    id: string
    buyerId: ObjectId (ref: Buyer)
    accepted: boolean (default: false)
    rejected: boolean (default: false)
    assignedAt: Date
    notes?: string
  }]
  soldTo: [{
    buyerId: ObjectId (ref: Buyer)
    createdAt: Date
    unit: number
  }]

  // Metadata
  industry?: string (indexed)
  leadSource?: string
  isManual?: boolean
  location?: {
    city?: string
    state?: string
    country?: string
    zipCode?: string
    address?: string
  }

  // Follow-up Tracking
  followUps: [{
    date: Date
    method: "call" | "email" | "sms"
    outcome: string
  }]

  // Timestamps
  createdAt: Date (default: Date.now, indexed)
  updatedAt: Date
}
```

Indexes:

- `userId` (for seller queries)
- `email` (for duplicate checking)
- `status` (for filtering)

- `industry` (for matching)
- `createdAt` (for sorting)
- Compound: `{ userId: 1, status: 1 }`

3. Buyer Model

File: `models/buyer.ts`

```
{
  _id: ObjectId

  // Basic Information
  name: string (required)
  company?: string
  email: string (required)
  phone?: string

  // Priority & Distribution
  priority: number (1-10, default: 5) // For weighted distribution

  // Capacity Management
  maxLeadsPerDay?: number
  currentLeadsToday: number (default: 0)
  currentLeads: number (default: 0)
  qualificationScoreMinimum?: number (0-100)

  // Financial
  walletBalance: number (default: 0)
  walletUnit: number (default: 0)

  // Status & Preferences
  status: "new" | "active" | "inactive" | "suspended"
  preferredDistribution?: "automatic" | "manual" | "direct"
  leadTypes: ["exclusive" | "shared"] (default: ["shared"])
  maxPricePerLead?: number
  autoAcceptMatchingLeads: boolean (default: false)

  // Location Matching
  serviceLocations: [{
    city?: string
    state?: string
    country?: string
    zipCodes: string[]
    radius?: number // in miles
  }]
  locationMatchingStrict: boolean (default: false)

  // Operating Hours
  workingHours?: {
    start: string // "HH:mm"
    end: string // "HH:mm"
```

```
}
timezone?: string

// Criteria Sets (Advanced Filtering)
criteriaSets: [{
  _id: ObjectId
  name: string
  leadTypes: ["exclusive" | "shared"]
  locations: [{
    city?: string
    state?: string
    zipCodes: string[]
    radius?: number
  }]
  industries: string[]
  maxPrice?: number
  dailyLimit?: number
  excludedSources: string[]
  autoAccept: boolean
  isDefault: boolean
}]
activeCriteriaSetId?: ObjectId

// History & Performance
purchaseHistory: [{
  leadId: ObjectId (ref: Lead)
  date: Date
  amount: number
  unit: number
}]
paymentHistory: [{
  date: Date
  amount: number
  method: "stripe" | "paypal"
}]
feedback: [{
  rating: number (1-5)
  comment?: string
  leadId: ObjectId
  date: Date
}]

// Ownership
```

```
  registeredWith: ObjectId (ref: User - seller) (required, indexed)

  // Notifications
  notificationPreferences: ["email" | "sms" | "dashboard"]

  // Timestamps
  createdAt: Date
  updatedAt: Date
}
```

Indexes:

- `registeredWith` (for seller queries)
- `email`
- `status`
- Compound: `{ registeredWith: 1, status: 1 }`

4. Form Model

File: `models/formModel.ts`

```
{
  _id: ObjectId
  formId: string (unique UUID)

  // Ownership
  userId: ObjectId (ref: User) (required, indexed)

  // Form Configuration
  formName: string (required)
  leadSource?: string
  industry?: string

  // Field Definitions
  fields: [{
    id: string (UUID)
    type: "text" | "email" | "phone" | "select" | "checkbox" | "radio" | "textarea" | "number"
    label: string
    required: boolean
    options?: string[] // For select, checkbox, radio
    placeholder?: string
    validation?: {
      pattern?: string // regex
      min?: number
      max?: number
    }
  }]

  // Tracking
  submittedLeads: ObjectId[] (ref: Lead)

  // Styling (optional)
  styling?: {
    primaryColor?: string
    buttonText?: string
    successMessage?: string
  }

  // Timestamps
  createdAt: Date
  updatedAt: Date
}
```

Indexes:

- `formId` (unique)
- `userId`

5. Tier Model

File: `models/tierModel.ts`

```
{
  _id: ObjectId

  // Basic Info
  name: string (required)
  price: number (required)
  description: string

  // Stripe Integration
  stripePriceId?: string
  stripeProductId?: string

  // Features
  features: string[]
  tierType: "free" | "paid"
  tierUserType: "seller" | "business"

  // Pricing Options
  discountPercentage?: number
  discountedPrice?: number
  renewalPrice?: number
  annualPrice?: number

  // Feature Limits
  tierLimits: {
    leads: number
    forms: number
    buyers: number
    numbers: number
    twilioNumbers: number
    callSeconds: number
    exports: number
    imports: number
    liveSupport: boolean
    industries: number
  }

  // Display
  isActive: boolean
  highlight: boolean
  order: number

  // Timestamps
```

```
  createdAt: Date
  updatedAt: Date
}
```

Indexes:

- `isActive`
- `tierUserType`
- `order`

6. Transaction Model

File: `models/transactions.ts`

```
{
  _id: ObjectId

  // Transaction Type
  type: "lead_purchase" | "call_purchase" | "units_purchase" |
        "seller_income" | "seller_payout" | "refund" |
        "admin_adjustment" | "subscription_payment" | "subscription_renewal"

  // User Reference
  userId: ObjectId (ref: User) (indexed)

  // Financial Details
  amount: number (required)
  currency: string (default: "USD")
  previousBalance: number
  currentBalance: number

  // Type-specific Metadata
  metadata: {
    // Lead Purchase
    leadId?: ObjectId
    unitsPurchased?: number

    // Seller/Buyer References
    sellerId?: ObjectId
    buyerId?: ObjectId

    // Subscription
    tierId?: ObjectId
    tierName?: string
    tierType?: string
    subscriptionPlan?: string
    subscriptionDuration?: string
    userEmail?: string

    // Refund
    refundReason?: string

    // Admin
    adminNote?: string

    // Payout
    stripeTransferId?: string
```

```
    payoutId?: string
    transferVerified?: boolean
    transferAmount?: number
  }

  // Payment Gateway
  paymentGateway: "stripe" | "paypal" | "square" | "manual"
  gatewayTransactionId?: string

  // Status
  status: "pending" | "completed" | "failed" | "refunded"

  // Relations
  relatedInvoices: ObjectId[] (ref: Invoice)

  // Timestamps
  createdAt: Date (default: Date.now, indexed)
  updatedAt: Date
}
```

Indexes:

- `userId`
- `type`
- `status`
- `createdAt`
- Compound: `{ userId: 1, createdAt: -1 }`

7. Invoice Model

File: `models/invoices.ts`

```
{
  _id: ObjectId

  // Parties
  userId: ObjectId (ref: User - seller) (required)
  buyerId: ObjectId (ref: Buyer) (required)

  // Invoice Details
  invoiceNumber: string (unique)
  invoiceDate: Date (required)
  dueDate: Date (required)

  // Line Items
  lineItems: [{
    description: string
    quantity: number
    unitPrice: number
    tax: number (default: 0)
    total: number
  }]

  // Totals
  subtotal: number (required)
  tax: number (default: 0)
  taxRate: number (default: 0)
  discount: number (default: 0)
  discountPercent: number (default: 0)
  total: number (required)
  currency: string (default: "USD")

  // Payment Status
  isPaid: boolean (default: false)
  status: "draft" | "sent" | "viewed" | "paid" | "overdue" | "cancelled"
  paymentMethod?: "stripe" | "paypal" | "bank_transfer" | "check"
  paymentDate?: Date

  // Recurring Invoices
  recurringEnabled: boolean (default: false)
  recurringFrequency?: "monthly" | "quarterly" | "annually"
  nextRecurringDate?: Date
  recurringEndDate?: Date
  parentInvoiceId?: ObjectId (ref: Invoice)
```

```
  // Communication
  remindersSent: number (default: 0)
  lastReminderDate?: Date
  notes?: string
  termsConditions?: string
  pdfUrl?: string

  // Relations
  relatedTransactions: ObjectId[] (ref: Transaction)

  // Timestamps
  createdAt: Date
  updatedAt: Date
}
```

Indexes:

- `invoiceNumber` (unique)
- `userId`
- `buyerId`
- `status`
- `dueDate`

8. Call Model

File: `models/callModel.ts`

```
{
  _id: ObjectId

  // Parties
  userId: string (seller ID) (indexed)
  buyerId?: string
  leadId?: ObjectId (ref: Lead)

  // Call Details
  callSid: string (Twilio call identifier)
  from: string (caller phone)
  to: string (tracking number)
  answeredBy?: string (buyer phone)

  // Status
  callStatus: string (Twilio status)
  status: "completed" | "failed" | "no-answer" | "busy"
  callDuration: number (in seconds)

  // Recording
  recordingUrl?: string
  callRecorded: boolean (default: false)

  // Charging
  unitsCharged: number (default: 0)
  paymentStatus: "paid" | "refunded" | "pending_refund"

  // Forwarding Configuration
  forwardingType: "single_multiple" | "specific_lead"
  forwardingNumbers: string[]
  leadBuyers?: ObjectId[]
  industry?: string
  reassigned?: boolean

  // Feedback & Quality
  feedback?: {
    buyerRating: boolean | null // true = good, false = bad
    sellerApproved: boolean | null // seller approves refund
    sellerComment?: string
    refundAmount?: number
    refundedAt?: Date
  }
```

```
  // Timestamps
  createdAt: Date
  updatedAt: Date
}
```

Indexes:

- `userId`
- `buyerId`
- `leadId`
- `callSid`
- `createdAt`

9. Campaign Model

File: `models/campaign.ts`

```
{
  _id: ObjectId

  // Ownership
  userId: ObjectId (ref: User) (indexed)

  // Campaign Details
  name: string (required)
  description?: string

  // Scheduling
  startDate: Date (required)
  endDate?: Date
  budget?: number

  // Target Audience
  targetLeads: ObjectId[] (ref: Lead)

  // Status
  status: "active" | "completed" | "draft"

  // Performance Metrics
  performanceMetrics?: Record<string, unknown>

  // Timestamps
  createdAt: Date (default: Date.now)
  updatedAt: Date (default: Date.now)
}
```

Indexes:

- `userId`
- `status`

10. Email Campaign Model

File: `models/emailCampaign.ts`

```
{
  _id: ObjectId

  // Ownership
  userId: ObjectId (ref: User) (indexed)

  // Campaign Info
  name: string (required)
  templateId?: ObjectId
  segmentId?: ObjectId

  // Email Content
  subject: string (required)
  htmlContent: string (required)
  textContent?: string
  fromName: string (required)
  fromEmail: string (required)
  replyTo?: string

  // Scheduling
  schedule: {
    type: "immediate" | "scheduled" | "recurring"
    scheduledTime?: Date
    recurring?: {
      frequency: "daily" | "weekly" | "monthly"
      daysOfWeek?: number[]
      endDate?: Date
    }
  }

  // Status
  status: "draft" | "scheduled" | "sending" | "paused" | "completed"

  // Analytics
  analytics: {
    sent: number (default: 0)
    delivered: number (default: 0)
    opened: number (default: 0)
    clicked: number (default: 0)
    unsubscribed: number (default: 0)
    bounced: number (default: 0)
    complained: number (default: 0)
    conversions: number (default: 0)
```

```
  }

  // A/B Testing
  abTesting?: {
    enabled: boolean
    variant: "A" | "B"
    variantSubject?: string
    splitPercentage: number
  }

  // Timestamps
  createdAt: Date
  updatedAt: Date
}
```

11. SMS Campaign Model

File: `models/smsCampaign.ts`

```
{
  _id: ObjectId

  // Ownership
  userId: ObjectId (ref: User) (indexed)

  // Campaign Info
  name: string (required)
  templateId?: ObjectId
  segmentId?: ObjectId

  // Recipients
  recipients: [{
    phone: string (required)
    name?: string
    variables?: Record<string, string>
  }]

  // Message Content
  textContent: string (required, max: 160 characters)

  // Scheduling
  scheduleAt?: Date

  // Status
  status: "draft" | "scheduled" | "sending" | "paused" | "completed"

  // Statistics
  stats: {
    queued: number (default: 0)
    sent: number (default: 0)
    delivered: number (default: 0)
    failed: number (default: 0)
    clicks: number (default: 0)
    replies: number (default: 0)
    optOuts: number (default: 0)
  }

  // Timestamps
  createdAt: Date
  updatedAt: Date
}
```

12. Automation Workflow Model

File: `models/automationWorkflow.ts`

```
{
  _id: ObjectId

  // Ownership
  userId: ObjectId (ref: User) (indexed)

  // Workflow Details
  name: string (required)
  description?: string
  isActive: boolean (default: true)

  // Triggers
  triggers: [{
    type: "lead_received" | "lead_accepted" | "lead_qualified" | "scheduled" | "manual"
    conditions?: [{
      field: string
      operator: "equals" | "contains" | "greater_than" | "less_than" | "in_array" | "exists"
      value: string | number | string[] | boolean
    }]
  }]

  // Actions
  actions: [{
    type: "send_email" | "send_sms" | "create_notification" | "assign_buyer" | "update_lead"
    config: {
      templateId?: ObjectId
      subject?: string
      body?: string
      message?: string
      buyerId?: ObjectId
      delayMinutes?: number
      priority?: "low" | "normal" | "high"
    }
  }]

  // Rate Limiting
  maxExecutions?: number
  executionsCount: number (default: 0)
  executionResetTime?: Date
  cooldownMinutes?: number

  // Priority
  priority: "low" | "normal" | "high" (default: "normal")
```

```
  // Statistics
  totalExecutions: number (default: 0)
  successCount: number (default: 0)
  failureCount: number (default: 0)
  lastExecutedAt?: Date

  // Timestamps
  createdAt: Date
  updatedAt: Date
}
```

Indexes:

- `userId`
- `isActive`

13. Webhook Config Model

File: `models/webhookConfig.ts`

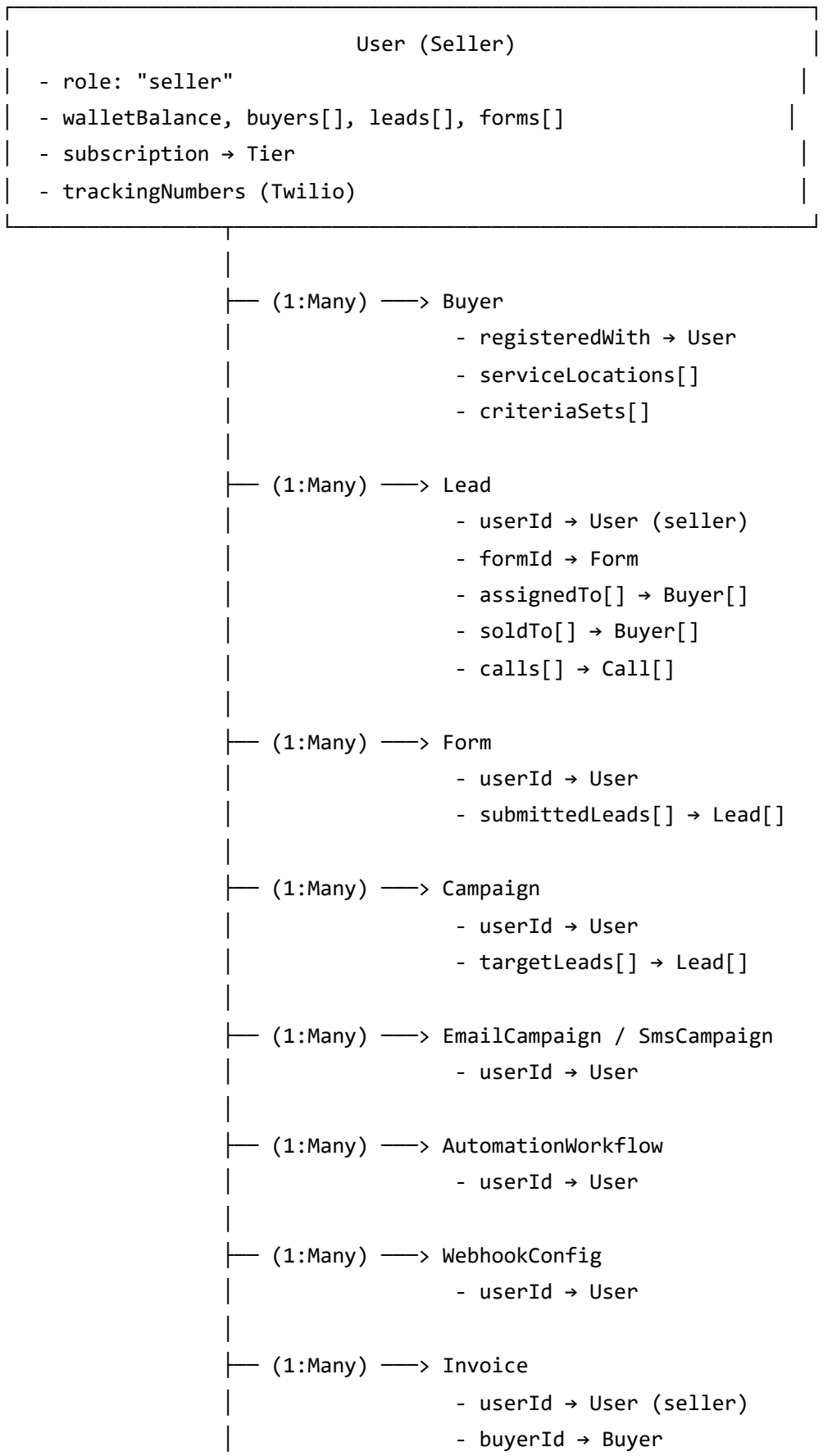
```
{
  _id: ObjectId

  // Ownership
  userId: ObjectId (ref: User) (indexed)

  // Webhook Details
  name: string (required)
  description?: string
  url: string (required)
  source: "zapier" | "hubspot" | "salesforce" |  "custom"

  // Security
  secret: string (generated)
  isActive: boolean (default: true)

  // Event Configuration
  events: {
    leadCreated: boolean (default: false)
    leadUpdated: boolean (default: false)
    leadQualified: boolean (default: false)
    leadAccepted: boolean (default: false)
    leadRejected: boolean (default: false)
    buyerAssigned: boolean (default: false)
    callCompleted: boolean (default: false)
    dealCreated: boolean (default: false)
    dealUpdated: boolean (default: false)
  }

  // Headers (for authentication)
  headers: Record<string, string>

  // Retry Configuration
  maxRetriesPerEvent: number (default: 3)
  retryDelaySeconds: number (default: 60)

  // Rate Limiting
  rateLimit?: {
    maxPerMinute: number
    maxPerHour: number
  }

  // Statistics
```

```
  totalDispatched: number (default: 0)
  successCount: number (default: 0)
  failureCount: number (default: 0)
  lastDispatchedAt?: Date
  lastErrorAt?: Date
  lastErrorMessage?: string

  // Dispatch Logs (last 100)
  dispatchLogs: [{
    timestamp: Date
    success: boolean
    statusCode?: number
    errorMessage?: string
    retryCount: number
    responseTime: number
  }]

  // Timestamps
  createdAt: Date
  updatedAt: Date
}
```

Indexes:

- userId
- isActive
- source

4. DATA RELATIONSHIP DIAGRAM



5. API ROUTES - ORGANIZED BY FUNCTIONALITY

Authentication & User Management

- `app/api/auth/[...nextauth]/route.ts` - NextAuth authentication handlers
- `app/api/users/route.ts` - Get users list (admin), create user
- `app/api/users/register/route.ts` - Register new user
- `app/api/users/profile/route.ts` - Update user profile
- `app/api/users/type/route.ts` - Set user type/role

Lead Management

- `app/api/leads/route.ts` - CRUD operations on leads
  - GET: Fetch leads with pagination & filters
  - POST: Create new lead
  - PUT: Update lead
  - DELETE: Delete lead
- `app/api/leads/available/route.ts` - Get marketplace leads (buyer view)
- `app/api/leads/exclusive/route.ts` - Get exclusive leads
- `app/api/leads/buyerleads/route.ts` - Get buyer-specific leads
- `app/api/leads/assignExclusiveLead/route.ts` - Assign exclusive lead to buyer(s)
- `app/api/leads/import/route.ts` - Bulk import leads

Buyer Management

- `app/api/buyers/route.ts` - CRUD operations on buyers
  - GET: Get all buyers for seller
  - POST: Create new buyer
  - PUT: Update buyer
  - DELETE: Delete buyer
- `app/api/buyers/[buyerId]/route.ts` - Get specific buyer details

Form Builder

- `app/api/form/route.ts` - CRUD operations on forms
  - GET: Get forms list
  - POST: Create new form
  - PUT: Update form
  - DELETE: Delete form
- `app/api/form/[formId]/route.ts` - Get form by ID
- `app/api/form/submit/route.ts` - Handle form submissions (creates lead)

Subscription & Tiers

- `app/api/tiers/route.ts` - Get all subscription tiers
- `app/api/tiers/[tierType]/route.ts` - Get tiers for user type
- `app/api/subscriptions/update/route.ts` - Update subscription
- `app/api/subscriptions/limits/route.ts` - Get subscription limits
- `app/api/subscriptions/check/route.ts` - Check subscription status

Payment Processing

Stripe

- `app/api/payments/stripe/stripecheckoutapi/route.ts` - Create checkout session
- `app/api/payments/stripe/stripewebhook/route.ts` - Stripe webhook handler
- `app/api/payments/stripe/process/route.ts` - Process Stripe payment
- `app/api/payments/stripe/transfer/route.ts` - Transfer funds to seller
- `app/api/payments/stripe/connect-account/route.ts` - Create Stripe Connect account
- `app/api/payments/stripe/onboard/route.ts` - Onboard seller to Stripe
- `app/api/payments/stripe/account-status/route.ts` - Check account status
- `app/api/payments/payout/stripe/route.ts` - Request Stripe payout

PayPal

- `app/api/payments/paypal/create/route.ts` - Create PayPal order
- `app/api/payments/paypal/capture/route.ts` - Capture PayPal payment
- `app/api/payments/paypal/paypal/route.ts` - Alternative capture endpoint
- `app/api/payments/paypal/webhook/route.ts` - PayPal webhook handler
- `app/api/payments/payout/paypal/route.ts` - Request PayPal payout
- `app/api/payments/paypal/client-id/route.ts` - Get PayPal client ID
- `app/api/payments/paypal/payouts/route.ts` - Alternative payout endpoint

General

- `app/api/payments/process/route.ts` - Generic payment processing
- `app/api/payments/transactions/route.ts` - Get transaction history
- `app/api/payments/status/route.ts` - Check payment status
- `app/api/payments/refund/route.ts` - Process refund

Invoices

- `app/api/invoices/route.ts` - Get all invoices, create invoice
- `app/api/invoices/[invoiceId]/route.ts` - Get, update invoice by ID

Marketing Campaigns

Email Marketing

- `app/api/marketing/email/campaigns/route.ts` - Get/create email campaigns
- `app/api/marketing/email/campaigns/[campaignId]/route.ts` - Get/update/delete campaign

- `app/api/marketing/email/campaigns/[campaignId]/analytics/route.ts` - Get campaign analytics
- `app/api/marketing/email/campaigns/[campaignId]/actions/route.ts` - Campaign actions (send, pause, resume)
- `app/api/marketing/email/templates/route.ts` - Get/create email templates
- `app/api/marketing/email/track/open/route.ts` - Track email opens
- `app/api/marketing/email/track/click/route.ts` - Track link clicks
- `app/api/marketing/email/unsubscribe/route.ts` - Unsubscribe handler

SMS Marketing

- `app/api/marketing/sms/campaigns/route.ts` - Get/create SMS campaigns
- `app/api/marketing/sms/campaigns/[campaignId]/route.ts` - Get/update/delete campaign
- `app/api/marketing/sms/campaigns/[campaignId]/actions/route.ts` - Campaign actions
- `app/api/marketing/sms/templates/route.ts` - Get/create SMS templates
- `app/api/marketing/sms/track/status/route.ts` - Track SMS status
- `app/api/marketing/sms/inbound/route.ts` - Handle inbound SMS

Automation Workflows

- `app/api/automation/workflows/route.ts` - Get/create workflows
- `app/api/automation/workflows/[workflowId]/route.ts` - Get/update/delete workflow

Call Tracking

- `app/api/calls/route.ts` - Get call tracking data
- `app/api/calls/twilio/route.ts` - Twilio webhook handler
- `app/api/calls/feedback/route.ts` - Submit call feedback

Integrations

Zapier

- `app/api/integrations/zapier/route.ts` - Get/create Zapier configurations
- `app/api/integrations/zapier/[id]/route.ts` - Update/remove integration

CRM Integrations

- `app/api/integrations/hubspot/*` - HubSpot integration endpoints
- `app/api/integrations/salesforce/*` - Salesforce integration endpoints

Notifications

- `app/api/notify/route.ts` - Get/create notifications
- `app/api/notify/[notificationId]/route.ts` - Get/update/delete notification

Settings

- `app/api/settings/route.ts` - Get/update user settings
- `app/api/settings/unitsettingapi/route.ts` - Get/create/update/delete unit pricing
- `app/api/settings/sellerlivechat/route.ts` - Get/update live chat config
- `app/api/settings/creditsetupapi/route.ts` - Configure payment credentials

Security

- `app/api/security/rotate-api-keys/route.ts` - Rotate API keys
- `app/api/security/audit-logs/route.ts` - Get audit logs

Admin

- `app/api/admin/*` - Admin-specific endpoints for user management, analytics

Support

- `app/api/support/tickets/route.ts` - Create/get support tickets
- `app/api/support/tickets/status/route.ts` - Update ticket status
- `app/api/support/tickets/followup/route.ts` - Add follow-up to ticket
- `app/api/support/help/faqs/route.ts` - Get/create/update/delete FAQs
- `app/api/support/help/videos/route.ts` - Get/upload/update/delete help videos

Miscellaneous

- `app/api/overview/route.ts` - Dashboard overview stats
- `app/api/health/route.ts` - Health check endpoint
- `app/api/csrf-token/route.ts` - Get CSRF token
- `app/api/searches/route.ts` - Save/get/delete saved searches
- `app/api/verifications/route.ts` - Email verification
- `app/api/cache/clear/route.ts` - Clear cache (admin)

Total API Endpoints: ~200+

File References: All routes are in `app/api/` directory following Next.js App Router conventions.

Document Summary

This document (1 of 3) covers:

- ✅ System architecture and technology stack
- ✅ Detailed user role descriptions with capabilities
- ✅ Complete database schema for all 13+ models
- ✅ Data relationship mapping
- ✅ API route organization (200+ endpoints)

Next Document: Business Workflows & Integration Capabilities
Final Document: Component Structure & Deployment Architecture